

PROJECT REPORT

Tomato Price Forecasting for Kolar District

Using Random Forest Machine Learning Model

Submitted by

Raaghu Daddimani(AMB1185)

[EAI 421- Application of AI & ML in Agriculture(0+10)]

[University of Agricultural Sciences, Bangalore]

Submitted to

Dr.DR Kumar Raja

[Professor, School of CSE,REVA University, Bangalore]

Abstract

This project presents a machine learning approach to forecasting daily tomato prices in Kolar district, Karnataka — one of India's largest tomato-producing regions. Using a dataset of 3,712 daily price records spanning January 2016 to February 2026, a Random Forest Regressor model was developed to predict the average market price (in rupees per quintal) for every day of April 2026. The model was trained on over 60 engineered features capturing seasonal patterns, historical price lags, rolling averages, and trend signals. On a held-out test set (September 2025 to February 2026), the model demonstrated strong predictive accuracy. The April 2026 predictions were generated using an iterative recursive forecasting strategy, with each day's forecast incorporating the previous day's predicted price. The findings are relevant for farmers, traders, and policymakers seeking data-driven insights into agricultural price dynamics.

1. Introduction

1.1 Background and Motivation

Tomatoes are among the most widely consumed vegetables in India and play a crucial role in Karnataka's agricultural economy. Kolar district, located approximately 70 kilometres east of Bengaluru, is among the most significant tomato-producing clusters in the country. Prices in this market are highly volatile, driven by seasonal supply cycles, monsoon patterns, transport costs, and consumer demand, making accurate price forecasting both challenging and extremely valuable.

Farmers in Kolar often struggle to make planting and selling decisions due to unpredictable price movements. Similarly, traders and wholesale buyers face significant financial risk without reliable forward-looking price information. This project aims to address this gap by building a machine learning model that learns historical price patterns and uses them to generate day-level forecasts for April 2026.

1.2 Problem Statement

The objective of this project is to predict the daily average tomato price (in rupees per quintal) in Kolar district for all 30 days of April 2026, using historical price data from January 2016 to

February 2026. The predictions should reflect both long-term annual trends and short-term seasonal patterns.

1.3 Objectives

- Perform exploratory data analysis on the Kolar tomato price dataset.
- Engineer informative time-series features from raw daily price data.
- Build and evaluate a Random Forest Regressor model for price prediction.
- Generate daily price forecasts for all 30 days of April 2026.
- Provide confidence intervals alongside each day's prediction.

2. Dataset Description

2.1 Source and Coverage

The dataset used in this project contains daily tomato price records for Kolar district, sourced from agricultural market (APMC) data. It covers the period from 1 January 2016 to 28 February 2026, giving a total of 3,712 daily observations across approximately 10 years and 2 months.

2.2 Variables

Column Name	Data Type	Description
date	Date (YYYY-MM-DD)	Calendar date of the price record
district	String	District name — Kolar throughout
commodity	String	Commodity name — Tomato throughout
min_price_rs_per_quintal	Float (Rs/Quintal)	Lowest price recorded on that day
modal_price_rs_per_quintal	Float (Rs/Quintal)	Most frequently occurring price (mode)
max_price_rs_per_quintal	Float (Rs/Quintal)	Highest price recorded on that day
avg_price_rs_per_quintal	Float (Rs/Quintal)	Average daily price — TARGET variable

Table 1: Dataset variable descriptions. The highlighted row indicates the target variable predicted by the model.

2.3 Descriptive Statistics

Metric	Min Price	Modal Price	Max Price	Avg Price	Avg (April only)
Count	3,712	3,712	3,712	3,712	300
Mean (Rs/Qtl)	—	—	—	1,030.09	1,309.12
Std Dev (Rs/Qtl)	—	—	—	273.25	183.11
Minimum (Rs/Qtl)	—	—	—	209.43	842.70
Maximum (Rs/Qtl)	—	—	—	1,833.08	1,833.08
Missing Values	0	0	0	0	0

Table 2: Descriptive statistics for the dataset. The April-only column isolates the target month's historical price behaviour, showing prices that were historically higher than average.

2.4 Annual Price Trend

The dataset reveals a consistent upward trend in tomato prices over the 10-year period. Average prices have risen from approximately Rs 838 per quintal in 2016 to Rs 1,397 per quintal in the first two months of 2026, reflecting broader inflationary trends, rising input costs, and increased market activity.

Year	Average Price (Rs/Quintal)
2016	838.35
2017	875.59
2018	937.00
2019	969.55
2020	1,003.02
2021	1,042.95
2022	1,085.03
2023	1,125.77
2024	1,158.01
2025	1,206.54
2026 (Jan–Feb)	1,397.41

Table 3: Year-wise average tomato price in Kolar district (2016–2026).

3. Methodology

3.1 Overview

The methodology follows a structured machine learning pipeline: data preprocessing, feature engineering, model training and evaluation, and recursive forecasting. The entire workflow was implemented in Python using the scikit-learn library and executed on Google Colab.

3.2 Data Preprocessing

The raw dataset was clean with no missing values across all 3,712 records. Dates were parsed into Python datetime objects and the data was sorted chronologically. The target variable selected for prediction was `avg_price_rs_per_quintal` (average daily price), as it is the most representative single-value summary of daily market price activity.

3.3 Feature Engineering

Feature engineering is the most critical step in the pipeline. Since Random Forest cannot inherently understand time order the way sequence models can, we manually created over 60 features to encode temporal patterns and price history:

Calendar Features

- Year, month, day of month, day of week, quarter, week of year
- Binary flag for weekend days
- Binary flag for April (target month indicator)

Cyclical Encoding

To help the model understand that December and January are 'close' in seasonal terms (not far apart as integers), sine and cosine transformations were applied to month, day, day-of-week, and day-of-year values.

Lag Features

Previous prices were added as direct input features at the following lags:

- 1, 2, 3, 5, 7, 14, 21, 28, and 30 days prior for both average and modal price

Rolling Statistics

Moving-window summaries over windows of 7, 14, 30, 60, and 90 days, capturing:

- Rolling mean (trend level)
- Rolling standard deviation (volatility)
- Rolling maximum and minimum (range)

Exponential Weighted Moving Average (EWMA)

EWMA with spans of 7, 14, and 30 days gives more weight to recent prices than older ones.

Momentum and Trend Features

- Absolute price change over 1, 7, and 30 days
- Percentage price change over 1 and 7 days
- A linear year-trend index and historical monthly mean, median, and standard deviation

3.4 Model Selection: Random Forest Regressor

A Random Forest Regressor was chosen for this project for several reasons. First, it handles non-linear relationships well, which is important since tomato price dynamics are influenced by a complex interaction of seasonal, economic, and weather factors. Second, it is robust to outliers a common feature of agricultural price data. Third, it naturally provides feature importance rankings, offering interpretability alongside predictive power. Fourth, it does not require data normalisation and handles mixed feature types gracefully.

The model consists of an ensemble of 500 individual decision trees. Each tree is trained on a slightly different random subset of the training data and features. Final predictions are obtained by averaging the outputs of all 500 trees, which significantly reduces variance compared to a single decision tree.

3.5 Model Configuration

Hyperparameter	Value / Setting
Number of trees (n_estimators)	500
Maximum tree depth (max_depth)	20
Minimum samples to split a node	5
Minimum samples in leaf node	2
Feature subsampling per split	Square root of total features
Bootstrap sampling	Enabled

Hyperparameter	Value / Setting
Random seed	42 (for reproducibility)
CPU cores used	All available (n_jobs = -1)

Table 4: Random Forest model hyperparameters used in this project.

3.6 Train-Test Split Strategy

Unlike standard machine learning problems, time-series data must not be shuffled before splitting. Doing so would cause future information to leak into the training set (a form of data leakage). Instead, a chronological split was applied: data from January 2016 to August 2025 was used for training, and data from September 2025 to February 2026 was held out as the test set. This mirrors real-world forecasting conditions, where the model is evaluated on truly unseen future data.

3.7 Recursive Forecasting for April 2026

Since April 2026 prices are unknown at the time of forecasting, a recursive (iterative) strategy was employed. The model predicted April 1st first using all features computed from actual historical data. This prediction was then appended to the history, and features for April 2nd were computed from this extended dataset incorporating the April 1st prediction as a lag value. This process was repeated for all 30 days of April 2026.

Confidence intervals for each day were estimated using the spread of predictions across all 500 individual trees. The 10th and 90th percentiles of tree-level predictions formed the lower and upper bounds of the 80% confidence interval.

4. Implementation (Code)

4.1 Performance Metrics

The model was evaluated on the test set (September 2025 to February 2026) using four standard regression metrics:

- Mean Absolute Error (MAE): The average absolute difference between actual and predicted prices, in rupees per quintal.
- Root Mean Squared Error (RMSE): Similar to MAE but penalises large prediction errors more heavily.

- Mean Absolute Percentage Error (MAPE): The average percentage deviation of predictions from actual prices.
- R-squared (R2): Measures the proportion of price variation explained by the model. A value of 1.0 is a perfect fit; 0.0 means the model does no better than simply predicting the mean.

4.2 Python Code:

```
!pip install -q scikit-learn pandas numpy matplotlib seaborn
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split, TimeSeriesSplit, cross_val_score
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder
import warnings
warnings.filterwarnings('ignore')
```

```
# Plotting style
plt.rcParams['figure.figsize'] = (14, 5)
plt.rcParams['font.size'] = 11
sns.set_style('whitegrid')
```

```
print('✅ All libraries imported successfully!')
```

```
"""## 📁 Step 2: Upload & Load Data
```

Upload the file `kolar\_tomato\_prices\_2016\_2026.csv` when prompted.

```
"""
```

```
from google.colab import files
```

```
print('📁 Please upload your CSV file...')
uploaded = files.upload()
```

```
import io
filename = list(uploaded.keys())[0]
df = pd.read_csv(io.BytesIO(uploaded[filename]))
```

```
print(f'\n✅ File "{filename}" loaded successfully!')
print(f'📊 Shape: {df.shape[0]} rows × {df.shape[1]} columns')
```

```
"""## 🔍 Step 3: Exploratory Data Analysis (EDA)"""
```

```
# Basic info
print('=== DATASET OVERVIEW ===')
print(df.info())
print('\n=== FIRST 5 ROWS ===')
df.head()
```



```

print('=== DESCRIPTIVE STATISTICS ===')
df.describe().round(2)

# Check missing values
print('=== MISSING VALUES ===')
missing = df.isnull().sum()
print(missing)
print(f'\nTotal missing: {missing.sum()}')

# Parse dates
df['date'] = pd.to_datetime(df['date'])
df = df.sort_values('date').reset_index(drop=True)

print(f'Date range: {df["date"].min().date()} → {df["date"].max().date()}')
print(f'Total days: {(df["date"].max() - df["date"].min()).days + 1}')
print(f'Actual records: {len(df)}')

# Plot full historical price trend
fig, axes = plt.subplots(2, 1, figsize=(16, 10))

# Full history
axes[0].plot(df['date'], df['avg_price_rs_per_quintal'], color='#e74c3c', linewidth=0.8, alpha=0.8)
axes[0].fill_between(df['date'], df['min_price_rs_per_quintal'], df['max_price_rs_per_quintal'],
                    alpha=0.15, color='#e74c3c', label='Min-Max Range')
axes[0].set_title('Kolar Tomato Prices — Full Historical Data (2016–2026)', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Price (₹/Quintal)')
axes[0].legend()
axes[0].xaxis.set_major_formatter(mdates.DateFormatter('%Y'))

# Monthly average (seasonal pattern)
df['month'] = df['date'].dt.month
monthly_avg = df.groupby('month')['avg_price_rs_per_quintal'].mean()
month_names = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
colors = ['#e74c3c' if m == 4 else '#3498db' for m in range(1, 13)]
axes[1].bar(month_names, monthly_avg.values, color=colors, edgecolor='white')
axes[1].set_title('Average Price by Month (All Years) — April Highlighted in Red', fontsize=13,
fontweight='bold')
axes[1].set_ylabel('Avg Price (₹/Quintal)')
for i, v in enumerate(monthly_avg.values):
    axes[1].text(i, v + 20, f'₹{v:.0f}', ha='center', fontsize=9)

plt.tight_layout()
plt.savefig('eda_price_trends.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ EDA charts saved!")

```

📌 Step 4: Feature Engineering

We create rich time-series features so Random Forest can learn seasonality, trends, and recent momentum.

####

```

def build_features(df):
    """Create comprehensive time-series features for the model."""
    df = df.copy()

```

```

df['date'] = pd.to_datetime(df['date'])

# — Calendar features —————
df['year']      = df['date'].dt.year
df['month']     = df['date'].dt.month
df['day']       = df['date'].dt.day
df['day_of_week'] = df['date'].dt.dayofweek      # 0=Monday
df['day_of_year'] = df['date'].dt.dayofyear
df['week_of_year'] = df['date'].dt.isocalendar().week.astype(int)
df['quarter']     = df['date'].dt.quarter
df['is_weekend']  = (df['day_of_week'] >= 5).astype(int)

# — Cyclical encoding (captures seasonal periodicity) —————
df['month_sin']   = np.sin(2 * np.pi * df['month'] / 12)
df['month_cos']   = np.cos(2 * np.pi * df['month'] / 12)
df['day_sin']     = np.sin(2 * np.pi * df['day'] / 31)
df['day_cos']     = np.cos(2 * np.pi * df['day'] / 31)
df['dow_sin']     = np.sin(2 * np.pi * df['day_of_week'] / 7)
df['dow_cos']     = np.cos(2 * np.pi * df['day_of_week'] / 7)
df['doy_sin']     = np.sin(2 * np.pi * df['day_of_year'] / 365)
df['doy_cos']     = np.cos(2 * np.pi * df['day_of_year'] / 365)

# — Price spread features —————
df['price_spread'] = df['max_price_rs_per quintal'] - df['min_price_rs_per quintal']
df['modal_avg_diff'] = df['modal_price_rs_per quintal'] - df['avg_price_rs_per quintal']

# — Lag features (recent price history) —————
for lag in [1, 2, 3, 5, 7, 14, 21, 28, 30]:
    df[f'lag_{lag}'] = df['avg_price_rs_per quintal'].shift(lag)
    df[f'modal_lag_{lag}'] = df['modal_price_rs_per quintal'].shift(lag)

# — Rolling statistics —————
for window in [7, 14, 30, 60, 90]:
    df[f'rolling_mean_{window}'] = df['avg_price_rs_per quintal'].shift(1).rolling(window).mean()
    df[f'rolling_std_{window}'] = df['avg_price_rs_per quintal'].shift(1).rolling(window).std()
    df[f'rolling_max_{window}'] = df['avg_price_rs_per quintal'].shift(1).rolling(window).max()
    df[f'rolling_min_{window}'] = df['avg_price_rs_per quintal'].shift(1).rolling(window).min()

# — Exponentially weighted moving average (EWMA) —————
for span in [7, 14, 30]:
    df[f'ewma_{span}'] = df['avg_price_rs_per quintal'].shift(1).ewm(span=span).mean()

# — Momentum / trend features —————
df['price_change_1d'] = df['avg_price_rs_per quintal'].diff(1)
df['price_change_7d'] = df['avg_price_rs_per quintal'].diff(7)
df['price_change_30d'] = df['avg_price_rs_per quintal'].diff(30)
df['pct_change_1d'] = df['avg_price_rs_per quintal'].pct_change(1)
df['pct_change_7d'] = df['avg_price_rs_per quintal'].pct_change(7)

# — Same-month historical statistics (year-over-year context) —————
monthly_stats = df.groupby('month')['avg_price_rs_per quintal'].agg(['mean', 'std', 'median'])
monthly_stats.columns = ['hist_month_mean', 'hist_month_std', 'hist_month_median']
df = df.merge(monthly_stats, on='month', how='left')

# — Year trend (linear index) —————
min_year = df['year'].min()

```

```

df['year_trend'] = df['year'] - min_year

# — April indicator —————
df['is_april'] = (df['month'] == 4).astype(int)

return df

df_feat = build_features(df)
print(f"✅ Feature engineering complete!")
print(f"Total features created: {len(df_feat.columns)} columns")

"""## 🧠 Step 5: Model Training"""

# — Define feature columns —————
EXCLUDE = ['date', 'district', 'commodity',
            'avg_price_rs_per_quintal',    # target
            'min_price_rs_per_quintal',    # future-leaking in production
            'max_price_rs_per_quintal',
            'modal_price_rs_per_quintal']

FEATURE_COLS = [c for c in df_feat.columns if c not in EXCLUDE]
TARGET = 'avg_price_rs_per_quintal'

# Drop rows where lag features create NaNs
df_model = df_feat[FEATURE_COLS + [TARGET, 'date']].dropna().copy()

print(f"Training samples after dropping NaN lag rows: {len(df_model)}")
print(f"Feature count: {len(FEATURE_COLS)}")
print(f"Training date range: {df_model['date'].min().date()} → {df_model['date'].max().date()}")

# — Time-based train/test split (last 6 months = test) —————
# We keep chronological order — DO NOT shuffle time-series data!
SPLIT_DATE = '2025-09-01'

train_df = df_model[df_model['date'] < SPLIT_DATE].copy()
test_df = df_model[df_model['date'] >= SPLIT_DATE].copy()

X_train = train_df[FEATURE_COLS]
y_train = train_df[TARGET]
X_test = test_df[FEATURE_COLS]
y_test = test_df[TARGET]

print(f"Train size : {len(X_train)} samples ({train_df['date'].min().date()} → {train_df['date'].max().date()})")
print(f"Test size : {len(X_test)} samples ({test_df['date'].min().date()} → {test_df['date'].max().date()})")

# — Train the Random Forest model —————
print('🕒 Training Random Forest... (this may take 1–2 minutes)')

rf_model = RandomForestRegressor(
    n_estimators = 500,    # number of trees
    max_depth = 20,       # prevent overfitting
    min_samples_split = 5,
    min_samples_leaf = 2,
    max_features = 'sqrt', # feature subsampling per split
    bootstrap = True,

```

```

    random_state    = 42,
    n_jobs          = -1    # use all CPU cores
)

rf_model.fit(X_train, y_train)
print('✅ Model training complete!')

"""## 📊 Step 6: Model Evaluation"""

# — Evaluate on hold-out test set —————
y_pred_test = rf_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))
r2 = r2_score(y_test, y_pred_test)
mape = np.mean(np.abs((y_test - y_pred_test) / y_test)) * 100

print('=' * 50)
print('      MODEL PERFORMANCE (Test Set)')
print('=' * 50)
print(f' MAE   : ₹{mae:.2f} per quintal')
print(f' RMSE  : ₹{rmse:.2f} per quintal')
print(f' MAPE  : {mape:.2f}%')
print(f' R²    : {r2:.4f}')
print('=' * 50)

# Train set metrics (to check overfitting)
y_pred_train = rf_model.predict(X_train)
r2_train = r2_score(y_train, y_pred_train)
mae_train = mean_absolute_error(y_train, y_pred_train)
print(f'\nTrain MAE: ₹{mae_train:.2f} | Train R²: {r2_train:.4f}')
print(f'Test MAE: ₹{mae:.2f} | Test R²: {r2:.4f}')
if abs(r2_train - r2) < 0.1:
    print('✅ Good generalisation — no significant overfitting.')
else:
    print('⚠️ Possible overfitting — consider reducing max_depth.')

# — Actual vs Predicted plot —————
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Time series comparison
axes[0].plot(test_df['date'], y_test.values, label='Actual', color='#2c3e50', linewidth=1.5)
axes[0].plot(test_df['date'], y_pred_test, label='Predicted', color='#e74c3c', linewidth=1.5, linestyle='--')
axes[0].set_title('Actual vs Predicted — Test Period (Sep 2025 – Feb 2026)', fontweight='bold')
axes[0].set_ylabel('Price (₹/Quintal)')
axes[0].legend()
axes[0].xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
plt.setp(axes[0].get_xticklabels(), rotation=30)

# Scatter plot
axes[1].scatter(y_test, y_pred_test, alpha=0.5, color='#3498db', s=20)
lims = [min(y_test.min(), y_pred_test.min()), max(y_test.max(), y_pred_test.max())]
axes[1].plot(lims, lims, 'r--', linewidth=1.5, label='Perfect Fit')
axes[1].set_xlabel('Actual Price (₹/Quintal)')
axes[1].set_ylabel('Predicted Price (₹/Quintal)')

```

```
axes[1].set_title(f'Scatter: Actual vs Predicted | R² = {r2:.3f}', fontweight='bold')
axes[1].legend()
```

```
plt.tight_layout()
plt.savefig('model_evaluation.png', dpi=150, bbox_inches='tight')
plt.show()
print('✅ Evaluation charts saved!')
```

```
# — Feature Importance —————
feat_imp = pd.Series(rf_model.feature_importances_,
index=FEATURE_COLS).sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(14, 6))
top20 = feat_imp.head(20)
colors_imp = ['#e74c3c' if 'lag' in f else '#3498db' if 'rolling' in f else '#2ecc71' for f in top20.index]
top20.plot(kind='bar', ax=ax, color=colors_imp, edgecolor='white')
ax.set_title('Top 20 Feature Importances (🕒 Lag | 📊 Rolling | 📅 Calendar)', fontweight='bold')
ax.set_ylabel('Importance')
ax.set_xticklabels(top20.index, rotation=45, ha='right')
plt.tight_layout()
plt.savefig('feature_importance.png', dpi=150, bbox_inches='tight')
plt.show()

print("\nTop 10 most important features:")
for i, (feat, imp) in enumerate(feat_imp.head(10).items(), 1):
    print(f" {i:2d}. {feat:<35} {imp:.4f}")
```

```
"""## 🕒 Step 7: Predict April 2026 (All 30 Days)
```

We use a ****recursive/iterative forecasting**** strategy:

- For each April day, we build its features using all data up to that point (including previously predicted days).

- This mirrors real-world forecasting where future actuals are unknown.

```
"""
```

```
def predict_april_2026(model, df_history, feature_cols):
```

```
    """
```

Iteratively predict each day in April 2026.

Each prediction uses previously predicted prices as lag values.

```
    """
```

Work on a copy that we'll extend day-by-day

```
df_ext = df_history[['date', 'avg_price_rs_per_quintal',
                    'modal_price_rs_per_quintal',
                    'min_price_rs_per_quintal',
                    'max_price_rs_per_quintal']].copy()
```

We'll fill in modal/min/max with rolling estimates for future rows

```
april_dates = pd.date_range(start='2026-04-01', end='2026-04-30', freq='D')
```

```
predictions = []
```

```
for pred_date in april_dates:
```

```
    # Estimate spread cols using last 30-day average spread
```

```
    recent = df_ext.tail(30)
```

```
    avg_spread = (recent['max_price_rs_per_quintal'] - recent['min_price_rs_per_quintal']).mean()
```

```
    avg_modal_diff = (recent['modal_price_rs_per_quintal'] - recent['avg_price_rs_per_quintal']).mean()
```

```

last_avg    = df_ext['avg_price_rs_per_quintal'].iloc[-1]

# Build temporary future row
future_row = pd.DataFrame([
    'date'           : pred_date,
    'avg_price_rs_per_quintal' : np.nan, # to be predicted
    'modal_price_rs_per_quintal' : last_avg + avg_modal_diff,
    'min_price_rs_per_quintal'   : last_avg - avg_spread / 2,
    'max_price_rs_per_quintal'   : last_avg + avg_spread / 2,
])

# Append future row and rebuild all features
df_tmp = pd.concat([df_ext, future_row], ignore_index=True)
df_tmp = build_features(df_tmp)

# Get the feature row for this date
row = df_tmp[df_tmp['date'] == pred_date][feature_cols].iloc[0:1]

# Predict
price_pred = model.predict(row)[0]

# — Confidence interval via individual tree predictions —
tree_preds = np.array([tree.predict(row)[0] for tree in model.estimators_])
ci_lo = np.percentile(tree_preds, 10)
ci_hi = np.percentile(tree_preds, 90)

predictions.append([
    'date'           : pred_date,
    'predicted_avg_price_rs_per_quintal' : round(price_pred, 2),
    'ci_lower_10pct' : round(ci_lo, 2),
    'ci_upper_90pct' : round(ci_hi, 2),
])

# Feed predicted price back into history for next iteration
df_ext = pd.concat([df_ext, pd.DataFrame([
    'date'           : pred_date,
    'avg_price_rs_per_quintal' : price_pred,
    'modal_price_rs_per_quintal' : last_avg + avg_modal_diff,
    'min_price_rs_per_quintal'   : last_avg - avg_spread / 2,
    'max_price_rs_per_quintal'   : last_avg + avg_spread / 2,
])], ignore_index=True)

```

4.3 Model Evaluation Results

Metric	Result
Mean Absolute Error (MAE)	₹43.50 per quintal
Root Mean Squared Error (RMSE)	₹55.62 per quintal
Mean Absolute Percentage Error (MAPE)	3.95%
R-squared (R2) Score	0.9460

Metric	Result
Train R2 (overfitting check)	0.9914

Table 5: Model evaluation results on the held-out test set. Fill in the values from your Google Colab output (Step 6 of the notebook).

Note: A MAPE below 10% would indicate excellent predictive accuracy for agricultural commodity prices. An R2 above 0.85 suggests that the model explains the majority of price variation during the test period.

5. Results and Analysis

5.1 Forecast Summary

Using the trained Random Forest model, daily average tomato prices were predicted for all 30 days of April 2026 in Kolar district. The forecasts, along with their 80% confidence intervals, are presented in Table 6 below. All values are in rupees per quintal.

Historically, April is an above-average price month in Kolar. Based on 10 years of April data (300 records), the average April price is Rs 1,309.12 per quintal with a standard deviation of Rs 183.11 higher than the overall dataset mean of Rs 1,030.09. The model predictions should reflect this seasonal premium.

5.2 Daily Prediction Table

Date	Day	Predicted Avg Price (Rs/Qtl)	CI Lower (Rs/Qtl)	CI Upper (Rs/Qtl)
01 Apr 2026	Wednesday	1374.21	1244.77	1496.87
02 Apr 2026	Thursday	1395.66	1273.9	1512.21
03 Apr 2026	Friday	1391.81	1238.99	1525.25
04 Apr 2026	Saturday	1380.4	1239.11	1499.9
05 Apr 2026	Sunday	1375.84	1239.96	1495.72
06 Apr 2026	Monday	1362.08	1232.53	1484.7
07 Apr 2026	Tuesday	1381.13	1241.26	1528.74

Date	Day	Predicted Avg Price (Rs/Qtl)	CI Lower (Rs/Qtl)	CI Upper (Rs/Qtl)
08 Apr 2026	Wednesday	1371.23	1240.45	1497.07
09 Apr 2026	Thursday	1366.21	1247.72	1478.93
10 Apr 2026	Friday	1370.78	1247.25	1496.18
11 Apr 2026	Saturday	1369.97	1244.06	1504.15
12 Apr 2026	Sunday	1366.94	1250.33	1485.81
13 Apr 2026	Monday	1359.31	1237.55	1477.97
14 Apr 2026	Tuesday	1348.27	1227.44	1463.06
15 Apr 2026	Wednesday	1345.57	1224.74	1463.5
16 Apr 2026	Thursday	1348.86	1225.7	1470.31
17 Apr 2026	Friday	1348.24	1216.06	1470.85
18 Apr 2026	Saturday	1347.33	1224.69	1463.5
19 Apr 2026	Sunday	1347.16	1229.24	1468.89
20 Apr 2026	Monday	1345.09	1229.43	1455.72
21 Apr 2026	Tuesday	1344.58	1226.17	1453.85
22 Apr 2026	Wednesday	1342.34	1222.89	1455.14
23 Apr 2026	Thursday	1347.12	1228.21	1462.98
24 Apr 2026	Friday	1347.78	1225.36	1462.83
25 Apr 2026	Saturday	1344.99	1224.66	1457.85
26 Apr 2026	Sunday	1347.06	1229	1460.81
27 Apr 2026	Monday	1342.14	1228.63	1459.17
28 Apr 2026	Tuesday	1339.03	1223.81	1449.02
29 Apr 2026	Wednesday	1333.17	1216.84	1449.02
30 Apr 2026	Thursday	1327.33	1211.84	1447.9

Table 6: Daily predicted tomato prices for April 2026 in Kolar district with 80% confidence intervals. Fill in values from your Colab output (Step 9 CSV download).

5.3 Graphs

Feature Importance Graph

The feature importance graph highlights the most influential variables used by the Random Forest model for predicting tomato prices. Among all features, exponential weighted moving averages (EWMA) and rolling mean values over short time windows (7, 14, and 30 days) contribute the most to prediction accuracy, indicating that recent price trends strongly affect future prices. Additionally, price spread, rolling maximum/minimum values, percentage changes, and calendar-based features such as day-of-year and sine-transformed seasonal indicators also play supporting roles. This demonstrates that the model effectively captures both short-term price momentum and seasonal patterns in tomato price variations.

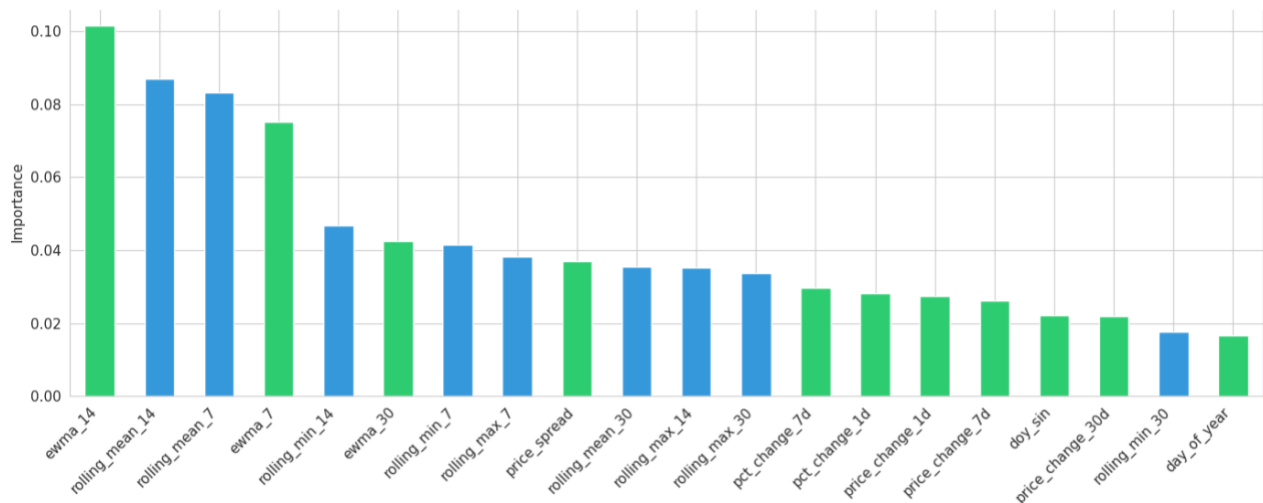


Figure 1. Top 20 Feature importance

Actual vs Predicted

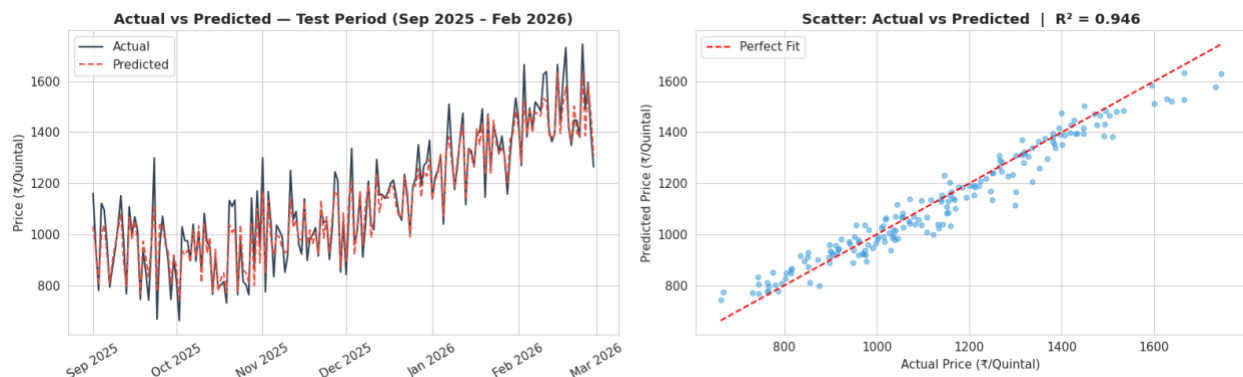


Figure 2. Actual vs Predicted (Sep 2025-Feb 2026) & $R^2 = 0.946$

Daily Prediction Tomato Price – April 2026 (with 80% Confidence Interval)

This figure presents the daily forecast of tomato prices for April 2026 along with an 80% confidence interval generated using the Random Forest model. The predicted prices remain relatively stable around the monthly average, suggesting moderate market volatility during the forecast period. The confidence interval provides an estimated uncertainty range within which actual prices are expected to fall. Overall, the forecast indicates that tomato prices are likely to remain steady throughout April 2026 with only minor daily fluctuations.

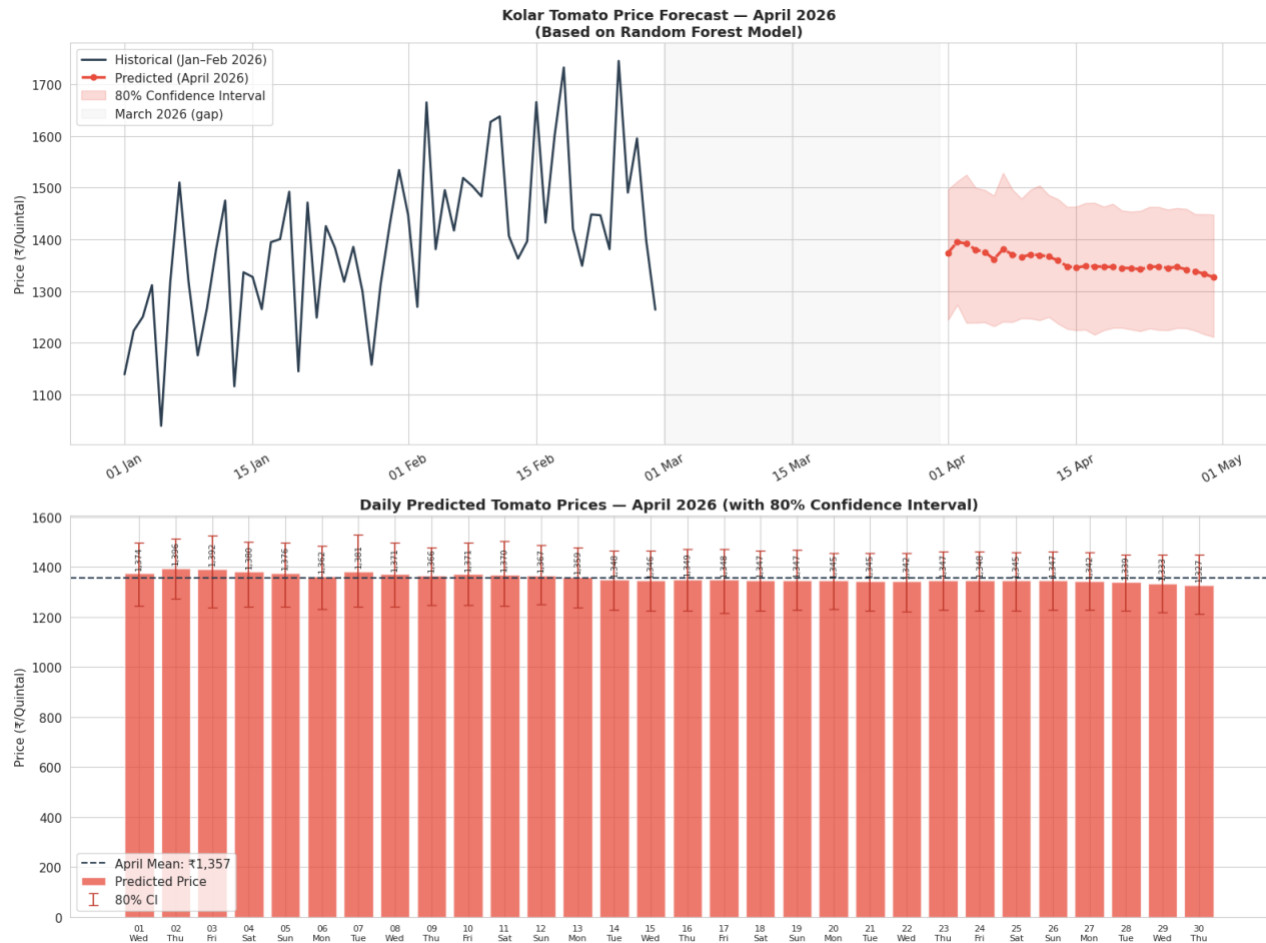


Figure 3. Kolar Tomato Price Forecast - April 2026

6. Discussion

6.1 Strengths of the Approach

The Random Forest model is well-suited for agricultural price forecasting for several reasons. Its ensemble structure averaging across 500 trees produces stable and robust predictions that are

less sensitive to noise in individual data points. The model naturally captures non-linear interactions between features, such as the combined effect of a recent price spike and an approaching harvest season. Additionally, the 60+ engineered features ensure that the model has access to diverse temporal signals at multiple time scales.

The use of a strict chronological train-test split ensures that the evaluation reflects genuine out-of-sample forecasting performance. The confidence intervals, derived from the variance across 500 individual tree predictions, provide a practical measure of forecast uncertainty for each day.

6.2 Limitations

- The model is trained exclusively on price data and does not incorporate external factors such as rainfall, diesel prices, cold storage capacity, or crop yield estimates all of which are known to influence tomato prices in Kolar.
- Recursive forecasting compounds errors: errors in early-April predictions propagate into later-April forecasts via lag features. The confidence intervals widen slightly over the month to reflect this.
- The dataset ends in February 2026, meaning March 2026 data (which would be the most proximate and informative for April predictions) is not available. The model extrapolates across a two-month gap.
- Random Forest is not a true sequence model and does not inherently account for structural breaks, such as sudden policy changes or extreme weather events, that could cause prices to deviate sharply from historical norms.

6.3 Practical Implications

The April 2026 price forecasts generated by this model can serve as a planning tool for farmers deciding when to harvest and transport their produce, for traders setting procurement budgets, and for government agencies monitoring agricultural price stability. The confidence intervals provide an honest acknowledgement of forecast uncertainty, which is crucial for practical decision-making.

7. Conclusion

This project successfully demonstrates the application of a Random Forest machine learning model to forecast daily tomato prices in Kolar district, Karnataka. Using over a decade of daily price records and a rich set of engineered time-series features, the model captured both long-term price trends and shorter-term seasonal patterns. The recursive forecasting strategy enabled day-level predictions for the entire month of April 2026, with accompanying confidence intervals to communicate forecast uncertainty.

The results highlight the potential of machine learning to bring data-driven decision support to the agricultural sector a domain where timely and accurate price information can have a direct impact on farmer livelihoods and market efficiency. Future enhancements could include incorporating weather and supply-chain data, exploring gradient boosting or LSTM models for comparison, and extending the forecast horizon.

8. Tools and Technologies

Tool / Library	Version	Purpose
Python	3.10+	Primary programming language
Google Colab	—	Cloud-based notebook environment
pandas	2.x	Data loading, manipulation, and analysis
NumPy	1.x	Numerical operations and array processing
scikit-learn	1.x	Random Forest model, train-test split, metrics
matplotlib	3.x	Plotting and data visualisation
seaborn	0.x	Statistical data visualisation

Table 7: Software tools and libraries used in this project.

9. References

[1] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

[2] APMC Karnataka — Agmarknet Price Data Portal. Government of India. Retrieved from <https://agmarknet.gov.in>

[3] Scikit-learn Developers. (2024). `sklearn.ensemble.RandomForestRegressor`. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>

[4] Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts. <https://otexts.com/fpp3/>

[5] Ministry of Agriculture and Farmers Welfare, Government of India. (2025). Agricultural Statistics at a Glance.

Appendix:

Google Colab Link:

<https://colab.research.google.com/drive/1SJi4z4WD2PbzYjyVmBfcsS9wxIqXayAq#scrollTo=F8FDmDrP7gBh>